

Project: Application Deployment

(Deploy the given React application to a production ready state)

Application: Clone the below mentioned repository and deploy the application (Run application in port 3000). Repo URL : <https://github.com/Vennilavanguvi/Brain-Tasks-App.git>

● **GitHub Repository:** <https://github.com/ersaurav88-devops/DevOps-Project-Brain-Tasks.git>

● **LoadBalancer ARN:** `arn:aws:elasticloadbalancing:us-east-1:060699023193:loadbalancer/aef99fdaff6c34e9e825b227eb2e2f44`

1. Clone the Repository

Clone Repository from GitHub

`git clone https://github.com/Vennilavanguvi/Brain-Tasks-App.git`

Move into project folder:

`cd Brain-Tasks-App`

2. Verify Repository Structure

The repository contains only production build files (dist/) and does not include source code or package.json.

Check files:

`dir`

3. Create Dockerfile

Create a file named:

Dockerfile

`FROM nginx:latest`

`COPY dist/ /usr/share/nginx/html`

`RUN sed -i 's/listen 80;/listen 3000;/' /etc/nginx/conf.d/default.conf`

`EXPOSE 3000`

`CMD ["nginx", "-g", "daemon off;"]`

Screenshot

- Dockerfile content

```
Dockerfile M X ! buildspec.yaml
Dockerfile
1 FROM nginx:latest
2 COPY dist/ /usr/share/nginx/html
3 RUN sed -i 's/listen      80;/listen      3000;/' /etc/nginx/conf.d/default.conf
4 EXPOSE 3000
5 CMD ["nginx", "-g", "daemon off;"]
```

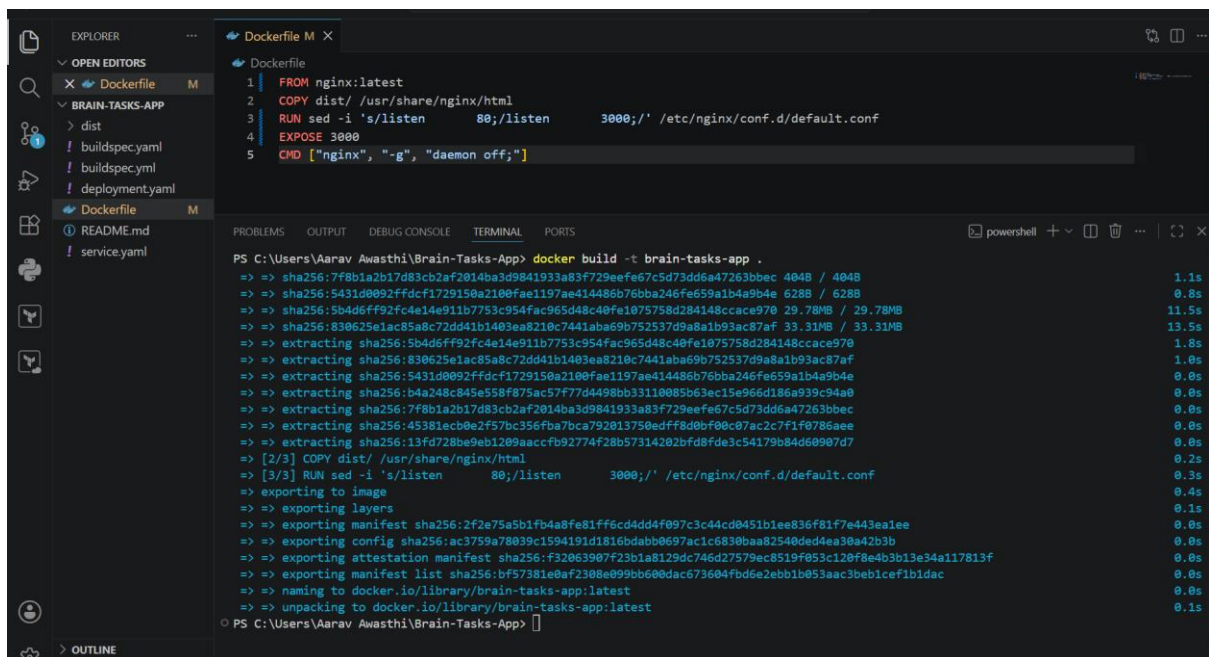
4. Build Docker Image

Run the following command:

`docker build -t brain-tasks-app .`

Screenshot

Successful docker build output



```
EXPLORER
OPEN EDITORS
Dockerfile M
BRAIN-TASKS-APP
dist
buildspec.yaml
buildspec.yaml
deployment.yaml
Dockerfile M
README.md
service.yaml

Dockerfile M X
Dockerfile
1 FROM nginx:latest
2 COPY dist/ /usr/share/nginx/html
3 RUN sed -i 's/listen      80;/listen      3000;/' /etc/nginx/conf.d/default.conf
4 EXPOSE 3000
5 CMD ["nginx", "-g", "daemon off;"]

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Aarav Awasthi\Brain-Tasks-App> docker build -t brain-tasks-app .
=> sha256:7f8b1a2b17d83cb2af2014ba3d9841933a83f729ee67c5d73dd6a47263bbec 4048 / 4048 1.1s
=> sha256:5431d0092ffdcf1729150a2100fae1197ae414486b76bba246fe659a1b4a9b4e 6288 / 6288 0.8s
=> sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 29.78MB / 29.78MB 11.5s
=> sha256:830625e1ac85a8c72dd41b1403ea8210c7441aba69b752537d9a8a1b93ac87af 33.31MB / 33.31MB 13.5s
=> extracting sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 1.8s
=> extracting sha256:830625e1ac85a8c72dd41b1403ea8210c7441aba69b752537d9a8a1b93ac87af 1.0s
=> extracting sha256:5431d0092ffdcf1729150a2100fae1197ae414486b76bba246fe659a1b4a9b4e 0.0s
=> extracting sha256:b4a248c845e58f875ac57f77d4498bb33110085b63ec15e66d186a939c34a0 0.0s
=> extracting sha256:7f8b1a2b17d83cb2af2014ba3d9841933a83f729ee67c5d73dd6a47263bbec 0.0s
=> extracting sha256:45381ecb0e2f57bc356fba7bca792013750edff8d0bf00c07ac2c7f1f0785aee 0.0s
=> extracting sha256:13fd728be9eb1209aaccfb92774f28b57314202bdf8fde3c54179b84d60907d7 0.0s
=> [2/3] COPY dist/ /usr/share/nginx/html 0.2s
=> [3/3] RUN sed -i 's/listen      80;/listen      3000;/' /etc/nginx/conf.d/default.conf 0.3s
=> exporting to image 0.4s
=> exporting layers 0.1s
=> exporting manifest sha256:2f2e75a5b1fb4a8fe81ff6cd4dd4f097c3c44cd0451b1ee836f817e443ea1ee 0.0s
=> exporting config sha256:ac3759a78039c1594191d1816bdabb0697ac1c6830baa82540ded4ea30a42b3b 0.0s
=> exporting attestation manifest sha256:f32063907f23b1a8129dc746d27579ec8519f053c120f8e4b3b13e34a117813f 0.0s
=> exporting manifest list sha256:bf57381e0af2308e099bb60dac673604fbd5e2ebb1b053aac3beb1cef1bdac 0.0s
=> naming to docker.io/library/brain-tasks-app:latest 0.0s
=> unpacking to docker.io/library/brain-tasks-app:latest 0.1s
PS C:\Users\Aarav Awasthi\Brain-Tasks-App>
```

5. Run Docker Container

Run container:

`docker run -d -p 3000:3000 --name brain-app brain-tasks-app`

Check running containers:

`docker ps`

6. Access Application

Open browser:

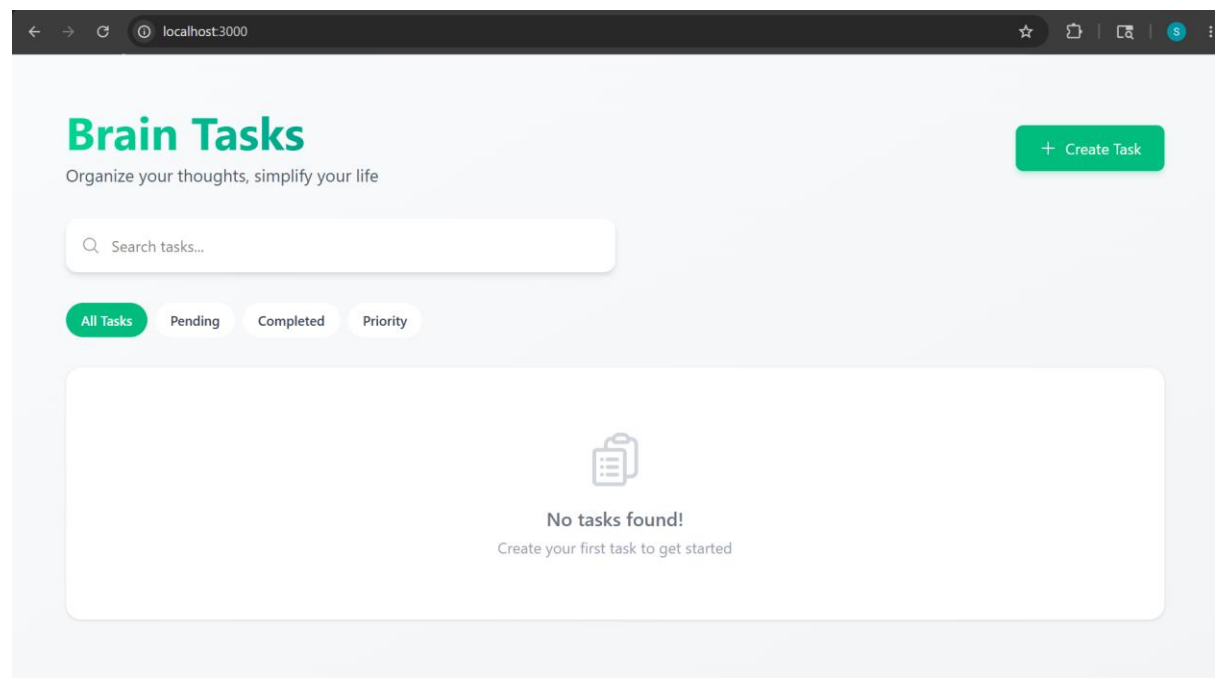
http://localhost:3000

Screenshot

Running Docker container

```
PS C:\Users\Aarav Awasthi\Brain-Tasks-App> docker run -d -p 3000:3000 --name brain-app brain-tasks-app
e8aae86365eb87d3e2a54b913408d495f98feaf7e8bc7cb4b43e8bbe0fe4d303
PS C:\Users\Aarav Awasthi\Brain-Tasks-App> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
e8aae86365eb   brain-tasks-app  "/docker-entrypoint..." About a minute ago Up About a minute   0.0.0.0:3000->3000/tcp, [::]:3000->3000/t
cp   brain-app
PS C:\Users\Aarav Awasthi\Brain-Tasks-App>
```

Application running in browser



7. Create AWS ECR Repository

Login to AWS Console.

Navigate to:

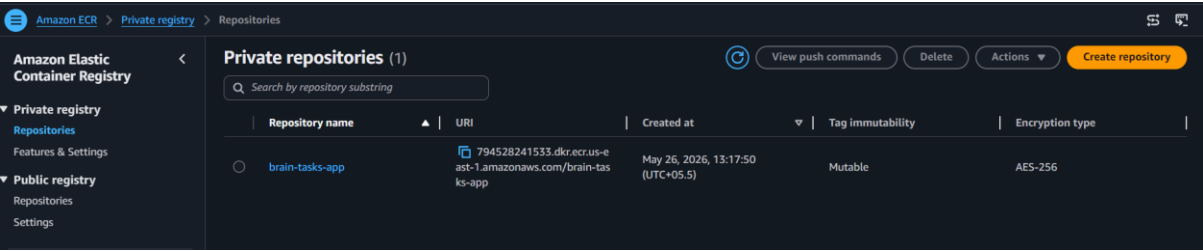
Amazon Elastic Container Registry (ECR)

Create repository:

brain-tasks-app

Screenshot

ECR repository created

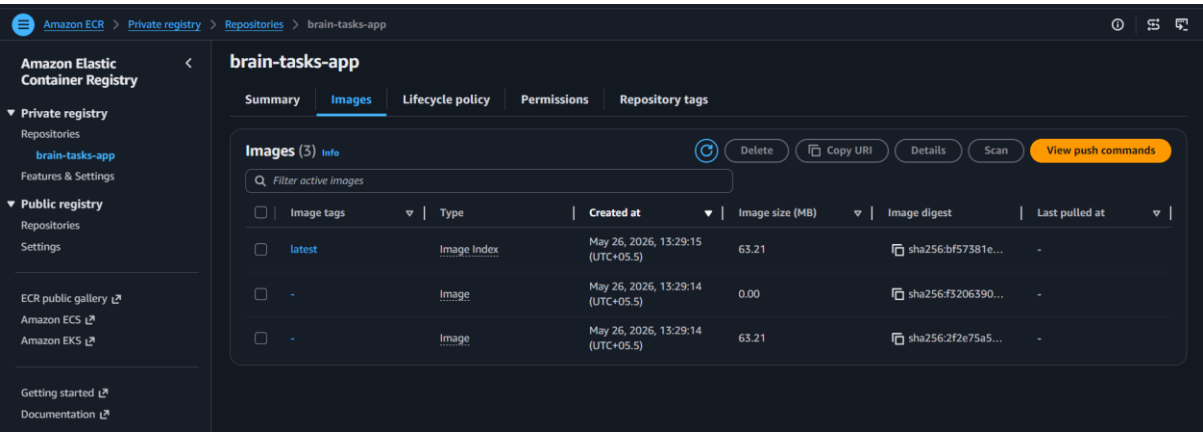
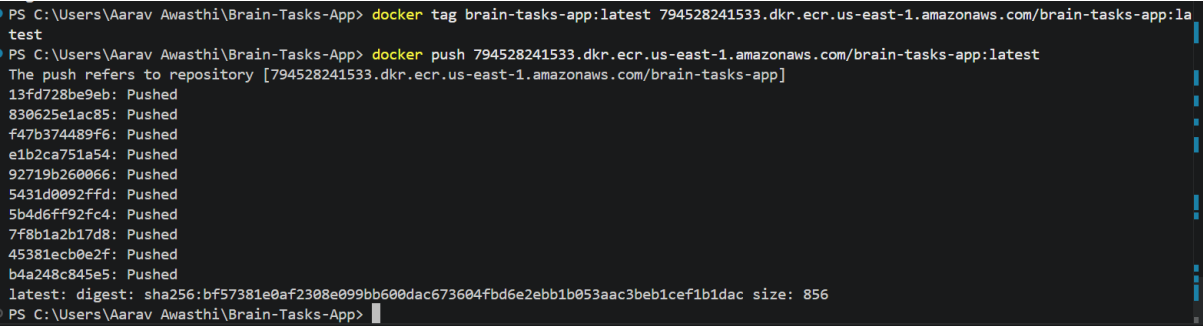


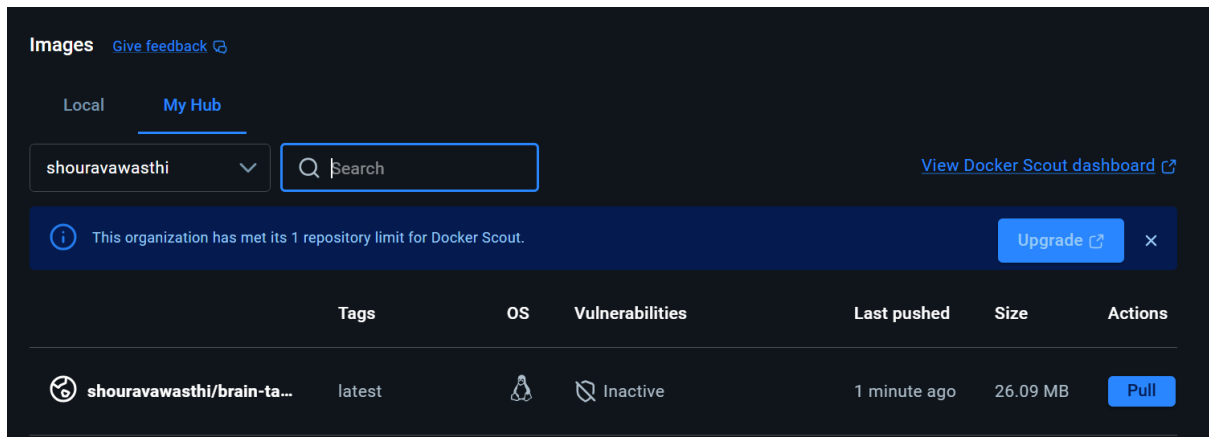
10. Push Docker Image to ECR

docker push 794528241533.dkr.ecr.us-east-1.amazonaws.com/brain-tasks-app:latest

Screenshot

Docker image successfully pushed to ECR





11. Setup AWS EKS Cluster

Create cluster using eksctl:

```
eksctl create cluster \
--name brain-cluster \
--region us-east-1 \
--nodegroup-name linux-nodes \
--node-type c7i-flex.large \
--nodes 2 \
--nodes-min 1 \
--nodes-max 3 \
--managed
```

12. Verify EKS Cluster

Run: kubectl get nodes

Screenshot

Running EKS cluster

```
ubuntu@ip-172-31-29-135:~$ eksctl get cluster
NAME          REGION    EKSCTL CREATED
brain-cluster  us-east-1  True
ubuntu@ip-172-31-29-135:~$
```

Worker nodes in Ready state

```
ubuntu@ip-172-31-29-135:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-13-48.ec2.internal      Ready    <none>   96s   v1.34.7-eks-7fcd7ec
ip-192-168-46-247.ec2.internal     Ready    <none>   97s   v1.34.7-eks-7fcd7ec
ubuntu@ip-172-31-29-135:~$
```

13. Create Kubernetes Deployment File

Create file: **deployment.yaml**

Content:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: brain-tasks-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: brain-tasks-app
  template:
    metadata:
      labels:
        app: brain-tasks-app
    spec:
      containers:
        - name: brain-tasks-app
          image: shouravawasthi/brain-tasks-app:latest
          ports:
            - containerPort: 3000
```

14. Create Kubernetes Service File

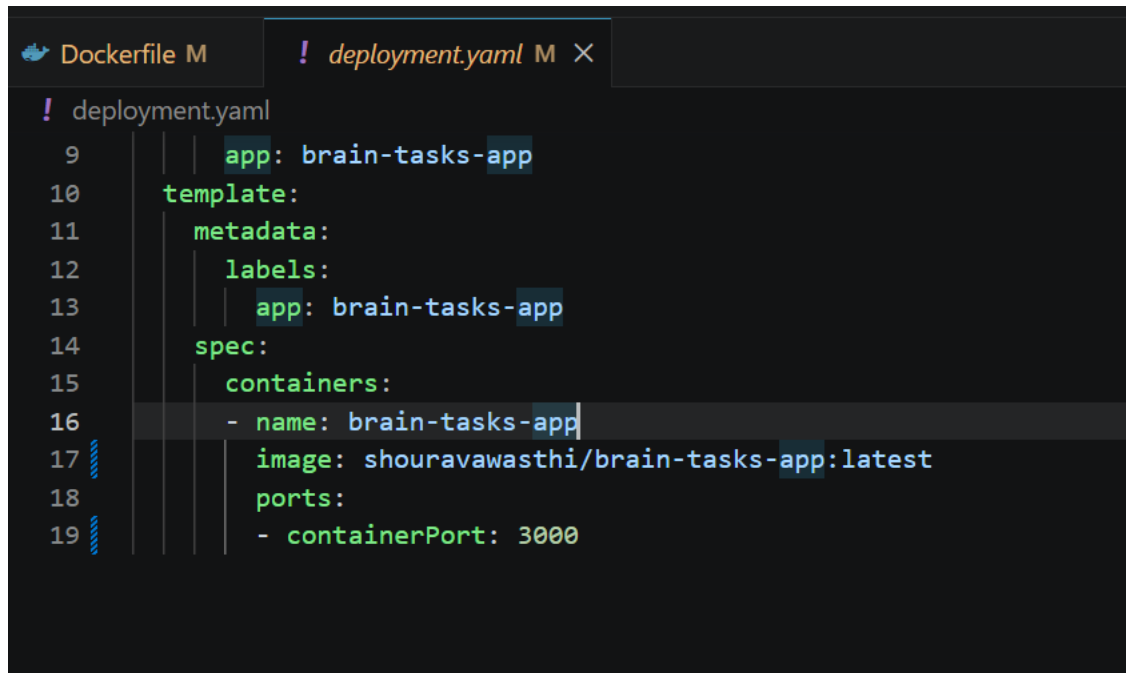
Create file: **service.yaml**

Content:

```
apiVersion: v1
kind: Service
metadata:
  name: brain-tasks-service
spec:
  type: LoadBalancer
  selector:
    app: brain-tasks-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3000
```

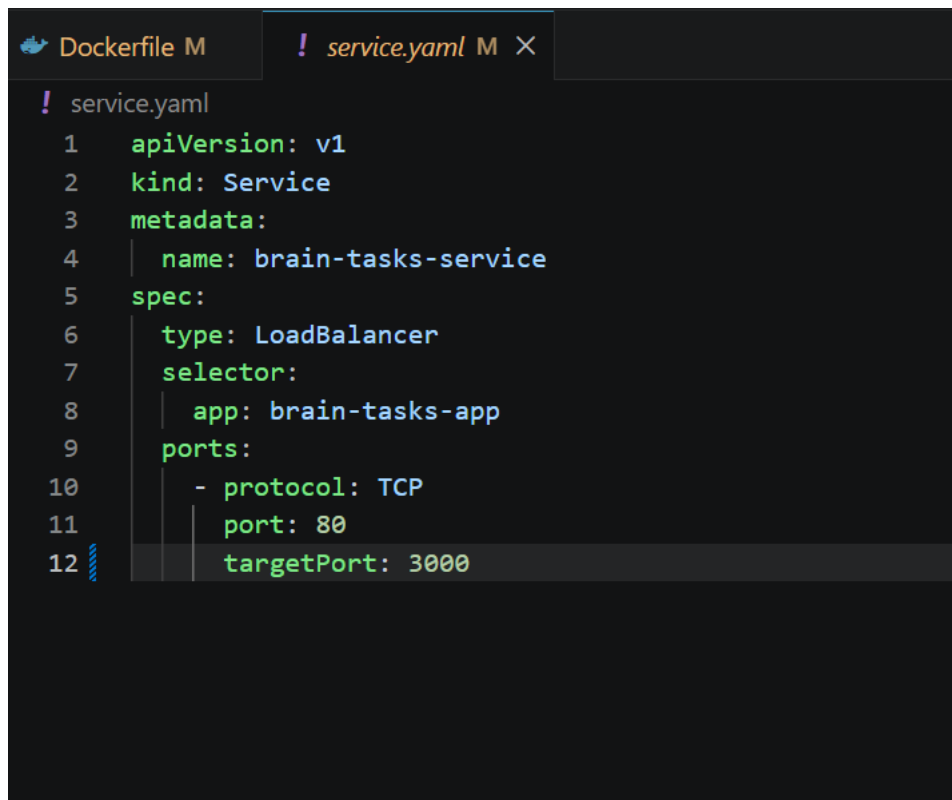
Screenshot

deployment.yaml



```
! deployment.yaml
 9   app: brain-tasks-app
10   template:
11     metadata:
12       labels:
13         app: brain-tasks-app
14     spec:
15       containers:
16       - name: brain-tasks-app
17         image: shouravawasthi/brain-tasks-app:latest
18         ports:
19         - containerPort: 3000
```

service.yaml



```
! service.yaml
 1  apiVersion: v1
 2  kind: Service
 3  metadata:
 4    name: brain-tasks-service
 5  spec:
 6    type: LoadBalancer
 7    selector:
 8      app: brain-tasks-app
 9    ports:
10      - protocol: TCP
11        port: 80
12        targetPort: 3000
```

15. Deploy Application to Kubernetes

Apply YAML files:

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

Verify resources:

```
kubectl get deployments
```

```
kubectl get pods
```

```
kubectl get svc
```

Screenshot

Take screenshot showing:

Deployment

```
ubuntu@ip-172-31-23-217:~/Brain-Tasks-App$ kubectl get deployments.apps
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
brain-tasks-deployment  2/2     2            2           29s
ubuntu@ip-172-31-23-217:~/Brain-Tasks-App$
```

- Running pods

```
ubuntu@ip-172-31-23-217:~/Brain-Tasks-App$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
brain-tasks-deployment-7667f78d76-cxhpz  1/1     Running   0          92s
brain-tasks-deployment-7667f78d76-kbcpw  1/1     Running   0          92s
ubuntu@ip-172-31-23-217:~/Brain-Tasks-App$
```

- LoadBalancer service

```
ubuntu@ip-172-31-23-217:~/Brain-Tasks-App$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP                                PORT(S)          AGE
brain-tasks-service  LoadBalancer  10.100.250.36  abf11a4fb09a64410a685bda5f62a45d-1106431807.us-east-1.elb.amazonaws.com  80:30996/TCP    3m3s
kubernetes          ClusterIP    10.100.0.1    <none>                                     443/TCP          29m
ubuntu@ip-172-31-23-217:~/Brain-Tasks-App$
```

16. Access Application via LoadBalancer

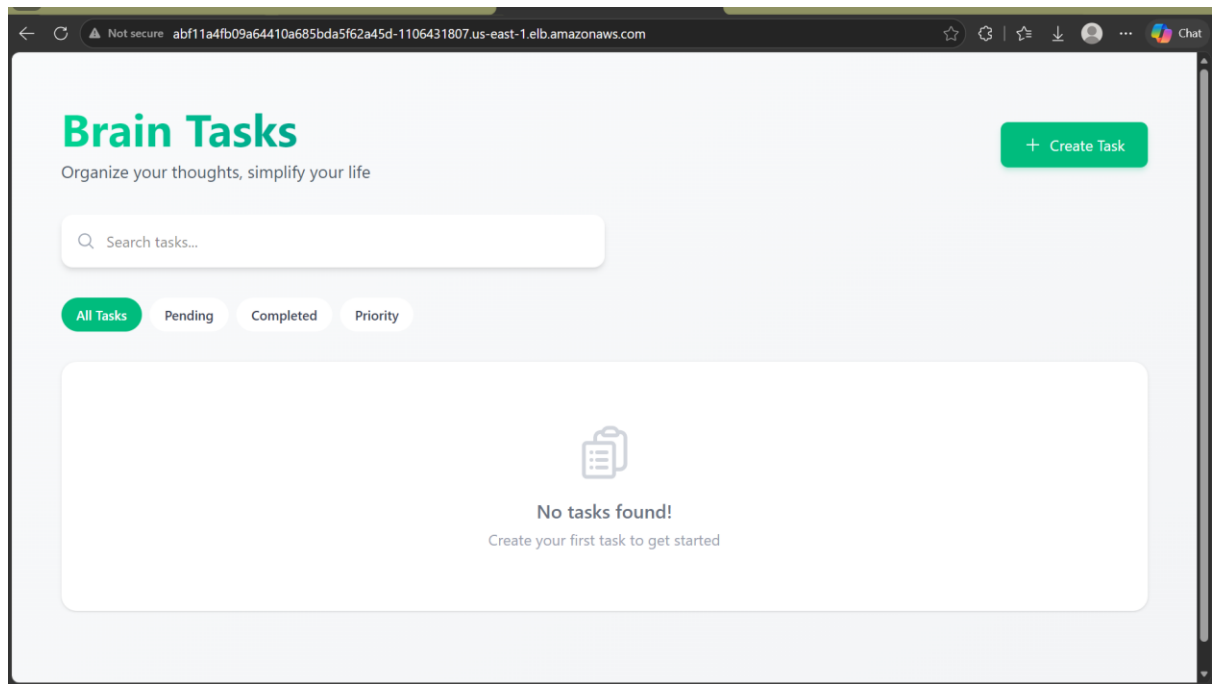
Copy EXTERNAL-IP from:

```
kubectl get svc
```

Open browser:

```
abf11a4fb09a64410a685bda5f62a45d-1106431807.us-east-1.elb.amazonaws.com
```

Screenshot : Application accessible from EKS LoadBalancer



17. Create buildspec.yml File

Create:

buildspec.yml

Add:

version: 0.2

env:

variables:

AWS_STS_REGIONAL_ENDPOINTS: regional

phases:

install:

commands:

- curl -LO "https://dl.k8s.io/release/v1.30.0/bin/linux/amd64/kubectl"
- install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
- chmod +x kubectl
- mv ./kubectl /usr/local/bin/kubectl
- kubectl version --client

pre_build:

commands:

- echo "Logging in to Amazon ECR..."
- aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 794528241533.dkr.ecr.us-east-1.amazonaws.com
- echo "Connecting to EKS Cluster..."
- aws eks update-kubeconfig --name brain-cluster --region us-east-1

- aws sts get-caller-identity
- kubectl get nodes

build:

commands:

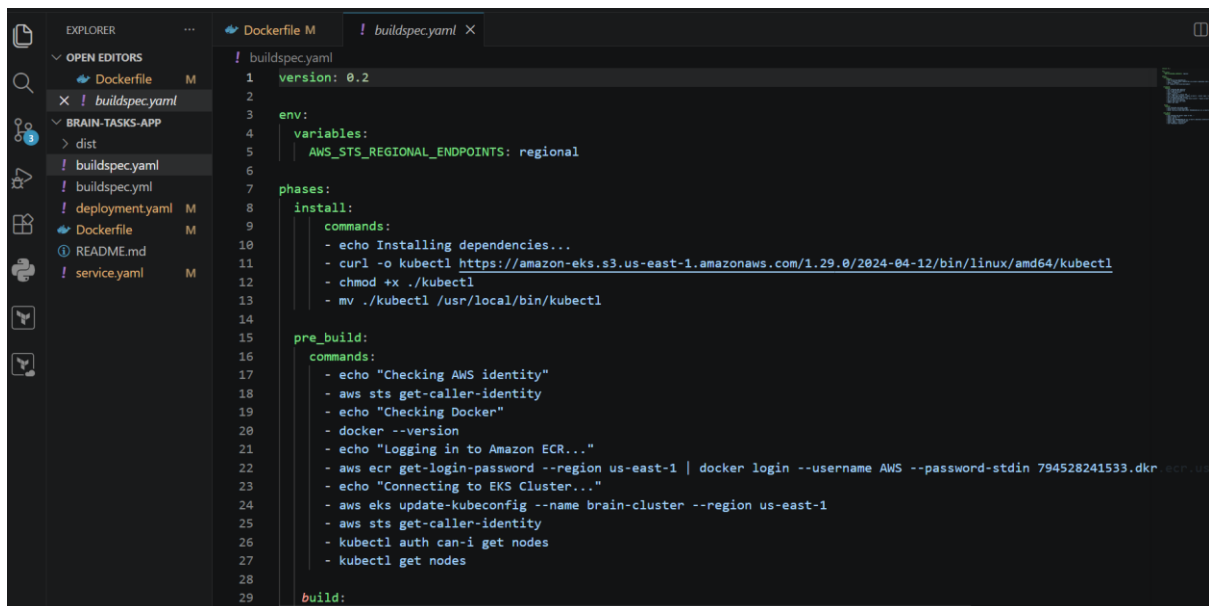
- echo "Building the Docker image..."
- docker build -t brain-tasks-app .
- docker tag brain-tasks-app:latest 794528241533.dkr.ecr.us-east-1.amazonaws.com/brain-tasks-app:latest

post_build:

commands:

- echo "Pushing the Docker image to ECR..."
- docker push 794528241533.dkr.ecr.us-east-1.amazonaws.com/brain-tasks-app:latest
- echo "Deploying to EKS..."
- kubectl apply -f deployment.yaml
- kubectl apply -f service.yaml
- echo "Deployment Complete!"

Screenshot buildspec.yml file



```
1 version: 0.2
2
3 env:
4   variables:
5     AWS_STS_REGIONAL_ENDPOINTS: regional
6
7 phases:
8   install:
9     commands:
10      - echo Installing dependencies...
11      - curl -o kubectl https://amazon-eks.s3.us-east-1.amazonaws.com/1.29.0/2024-04-12/bin/linux/amd64/kubectl
12      - chmod +x ./kubectl
13      - mv ./kubectl /usr/local/bin/kubectl
14
15   pre_build:
16     commands:
17      - echo "Checking AWS identity"
18      - aws sts get-caller-identity
19      - echo "Checking Docker"
20      - docker --version
21      - echo "Logging in to Amazon ECR..."
22      - aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 794528241533.dkr.ecr.us-east-1.amazonaws.com
23      - echo "Connecting to EKS Cluster..."
24      - aws eks update-kubeconfig --name brain-cluster --region us-east-1
25      - aws sts get-caller-identity
26      - kubectl auth can-i get nodes
27      - kubectl get nodes
28
29   build:
```

18. Create AWS CodeBuild Project

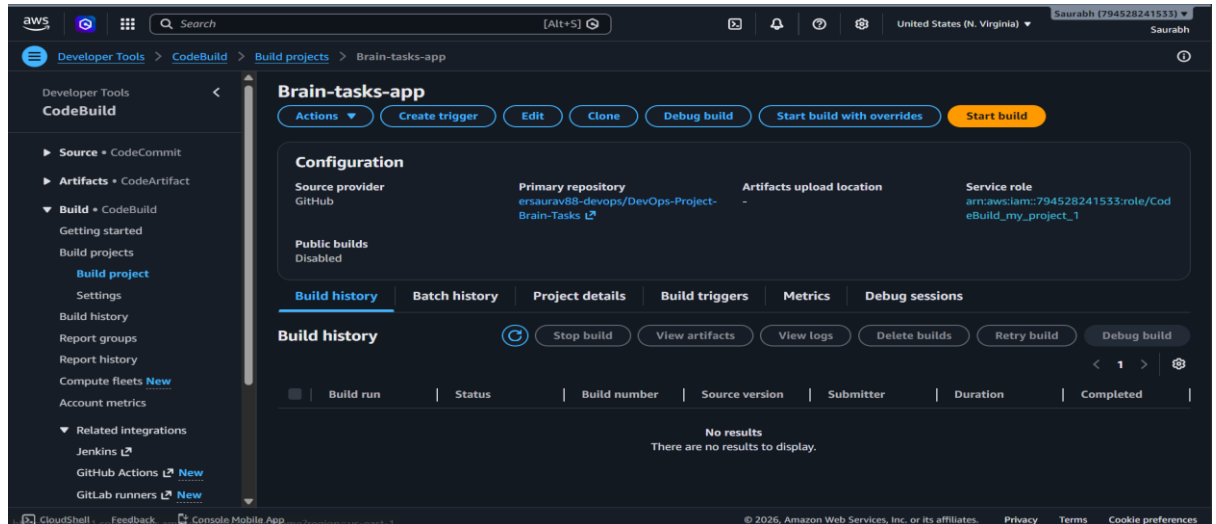
Configuration:

- Source Provider: GitHub
- Environment: Managed Image
- OS: Ubuntu / Amazon Linux

- Privileged Mode: Enabled

Screenshot:

- CodeBuild project configuration



19. Push Project to GitHub using CLI

Initialize repository:

```
git init
```

```
git add .
```

```
git commit -m "Initial deployment setup"
```

Push code:

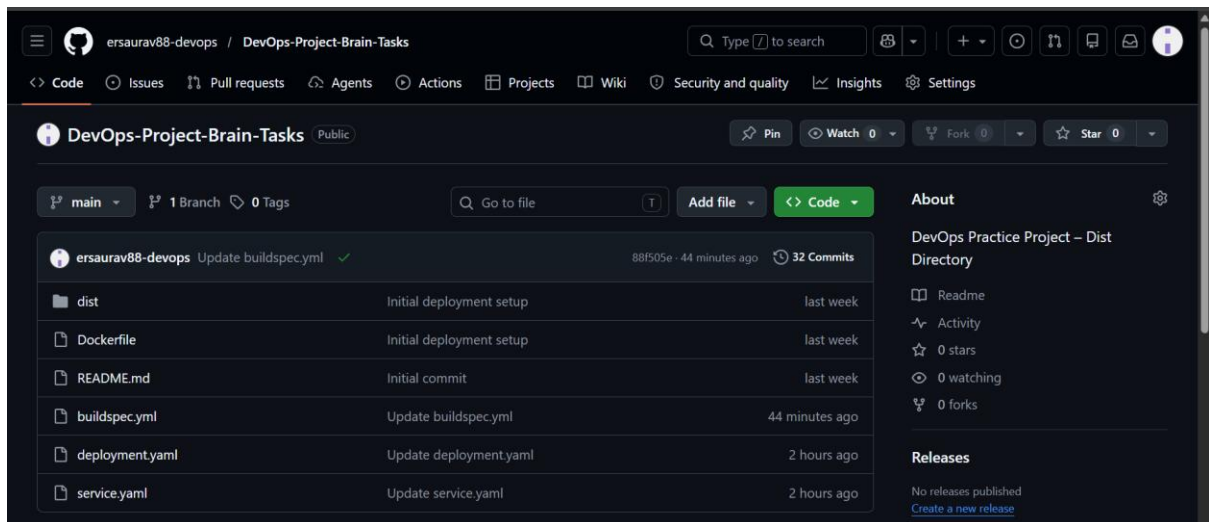
```
git push -u origin main
```

Screenshot

Git push successful

```
PS C:\Users\Aarav Awasthi\Brain-Tasks-App> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 6 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 100.09 KiB | 33.36 MiB/s, done.
Total 8 (delta 1), reused 6 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ersaurav88-devops/DevOps-Project-Brain-Tasks.git
 9e57ecb..87416c3 main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Aarav Awasthi\Brain-Tasks-App>
```

Repository visible on GitHub



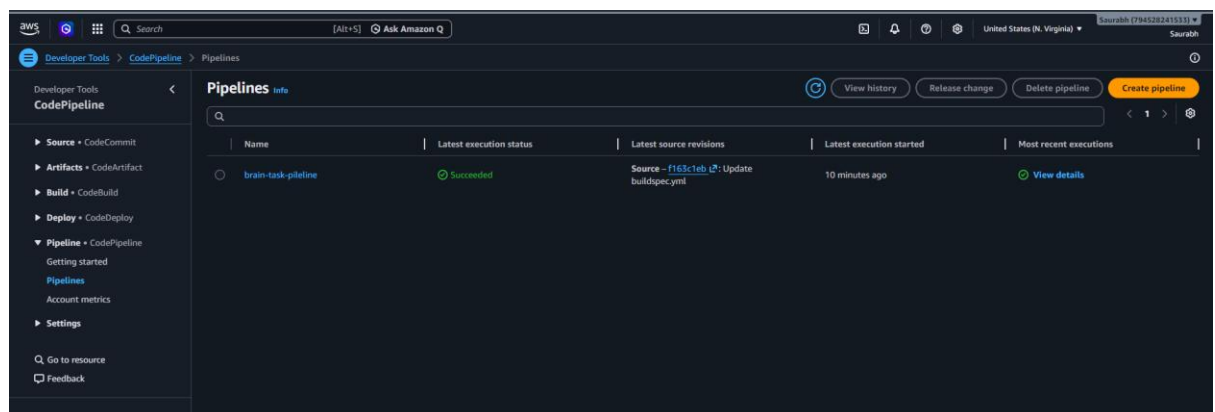
20. Create AWS CodePipeline

Pipeline Stages:

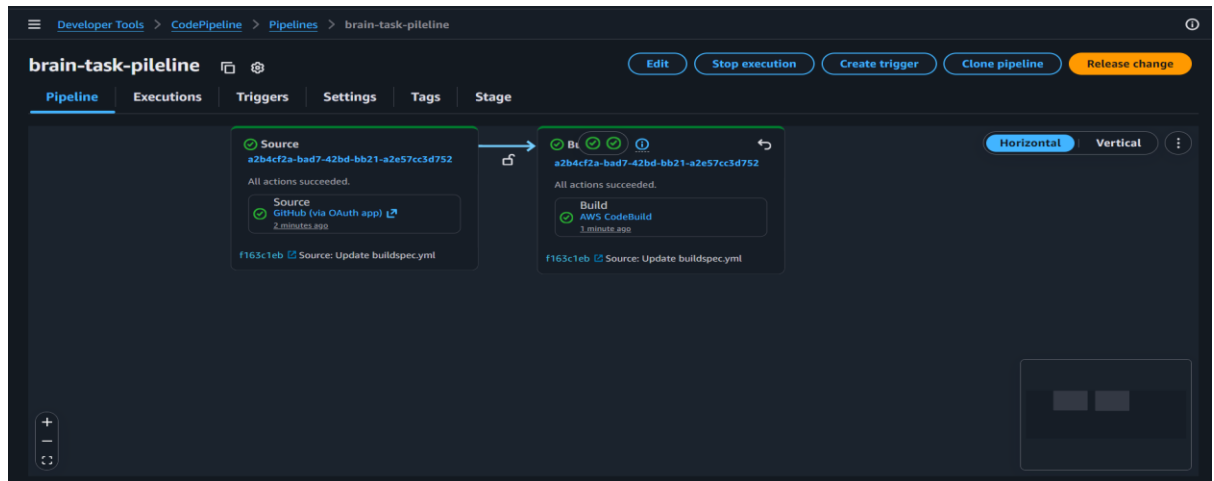
1. Source → GitHub
2. Build → AWS CodeBuild
3. Deploy → Amazon EKS

Screenshot

- CodePipeline stage



- Successful execution



21. Monitoring using CloudWatch Logs

Monitor:

- CodeBuild logs

```

157 ad8253b95bde: Layer already exists
158 5519239b187c: Layer already exists
159 906af8ee0f8a: Layer already exists
160 9a2cbc774b1f: Layer already exists
161 2d9b3b001567: Layer already exists
162 2b05bd5bca95: Layer already exists
163 b910427bd94b: Layer already exists
164 29df493baa13: Layer already exists
165 e9006114b296: Pushed
166 latest: digest: sha256:71cf1540e6d256bc05d91d091f900a3e40f07c8fb6c991a2b3eb7125a2b289c3 size: 2199
167
168 [Container] 2026/05/26 09:54:35.492843 Running command echo "Deploying to EKS..."
169 Deploying to EKS...
170
171 [Container] 2026/05/26 09:54:35.501204 Running command kubectl apply -f deployment.yaml
172 deployment.apps/brain-tasks-deployment unchanged
173
174 [Container] 2026/05/26 09:54:36.472035 Running command kubectl apply -f service.yaml
175 service/brain-tasks-service unchanged
176
177 [Container] 2026/05/26 09:54:37.508298 Running command echo "Deployment Complete!"
178 Deployment Complete!
179
180 [Container] 2026/05/26 09:54:37.516848 Phase complete: POST_BUILD State: SUCCEEDED
181 [Container] 2026/05/26 09:54:37.516867 Phase context status code: Message:
182 [Container] 2026/05/26 09:54:37.579968 Set report auto-discover timeout to 5 seconds
183 [Container] 2026/05/26 09:54:37.580022 Expanding base directory path: .
184 [Container] 2026/05/26 09:54:37.583662 Assembling file list
185 [Container] 2026/05/26 09:54:37.583677 Expanding .
186 [Container] 2026/05/26 09:54:37.587332 Expanding file paths for base directory .
187 [Container] 2026/05/26 09:54:37.587347 Assembling file list
188 [Container] 2026/05/26 09:54:37.587350 Expanding **/*
189 [Container] 2026/05/26 09:54:37.591427 No matching auto-discover report paths found
190 [Container] 2026/05/26 09:54:37.591456 Report auto-discover file discovery took 0.011487 seconds
191 [Container] 2026/05/26 09:54:37.591472 Phase complete: UPLOAD_ARTIFACTS State: SUCCEEDED
192 [Container] 2026/05/26 09:54:37.591481 Phase context status code: Message:
193

```

Screenshot CloudWatch Logs

The screenshot shows the AWS CloudWatch console interface. The left sidebar contains navigation links for CloudWatch, Log management, and various monitoring tools. The main content area displays the 'Log group details' for the log group `/aws/codebuild/Bain-Task-App-Build`. The details include the log class (Standard), ARN, creation time (23 hours ago), retention (Never expire), account retention policy, and stored bytes. There are also sections for Metric filters, Subscription filters, Contributor Insights rules, Sensitive data count, Custom field indexes, Transformer, Anomaly detection, and Bearer token authentication. Below the details, there is a 'Log streams' section showing a list of log streams with their last event times.

Log stream	Last event time
f969a408-6a33-4ef5-93a9-08c8da4e3963	2026-05-22 04:53:00 (UTC)
4f5038f3-a187-4bd2-8c21-e3526e3be89a	2026-05-22 04:52:25 (UTC)
af545417-bd18-4c14-83f0-3afe59bf645d	2026-05-22 04:48:24 (UTC)
b1a6fcc6-c51a-4372-bd0a-424aa754d7be	2026-05-22 04:40:48 (UTC)

Build logs

The screenshot shows the AWS CloudWatch console interface, specifically the 'Log events' section for the log group `/aws/codebuild/Bain-Task-App-Build`. The interface displays a list of log events with columns for Timestamp and Message. The events show the deployment process, including running commands, deploying to EKS, and completing the deployment. The messages are filtered to show only the events related to the deployment process.

Timestamp	Message
2026-05-22T04:53:18.351Z	[Container] 2026/05/22 04:53:17.645346 Running command echo "Deploying to EKS..."
2026-05-22T04:53:18.351Z	Deploying to EKS...
2026-05-22T04:53:18.351Z	[Container] 2026/05/22 04:53:17.653328 Running command kubectl apply -f deployment.yml
2026-05-22T04:53:18.851Z	deployment.apps/brain-tasks-deployment unchanged
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.598195 Running command echo "Deployment Complete!"
2026-05-22T04:53:18.851Z	Deployment Complete!
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.609642 Phase complete: POST_BUILD State: SUCCEEDED
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.609673 Phase context status code: Message:
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.668178 Set report auto-discover timeout to 5 seconds
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.668217 Expanding base directory path: .
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.671854 Assembling file list
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.671666 Expanding .
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.675133 Expanding file paths for base directory .
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.675146 Assembling file list
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.675349 Expanding */*
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.678788 No matching auto-discover report paths found
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.678809 Report auto-discover file discovery took 0.018631 seconds
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.678823 Phase complete: UPLOAD_ARTIFACTS State: SUCCEEDED
2026-05-22T04:53:18.851Z	[Container] 2026/05/22 04:53:18.678832 Phase context status code: Message: